

0. 序言

本文以一个简单的todoList前后端项目为例，进行devcloud平台的使用说明

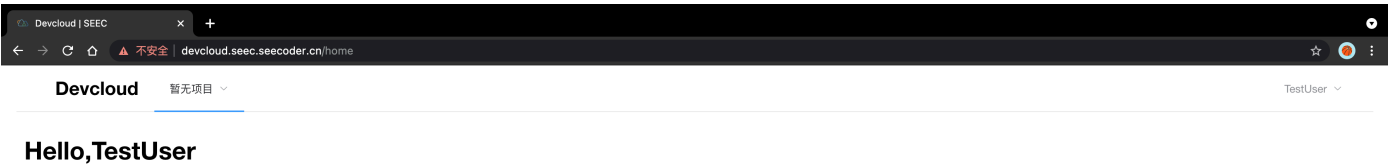
地址为：<https://devcloud.seec.seecoder.cn>

另：建议使用Chrome浏览器打开，使用其他浏览器侧边栏样式会有变化

1. 项目初始化

1.1 项目概览

注册账号并登录后的界面如下（因为还未创建项目，所以是空的）：



1.2 创建项目

只需小组其中一个成员来创建，其他成员已邀请的形式拉入项目，无需额外创建项目

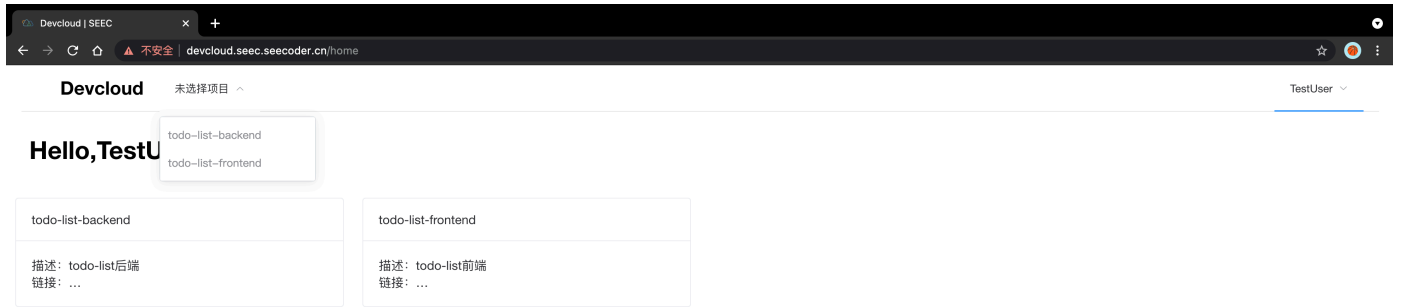
点击右上角id右边的下拉框，选择创建项目

输入项目名称，项目名称只能为以小写英文字母开头，且由英文字母和‘-’组成的字符串，长度为3-30



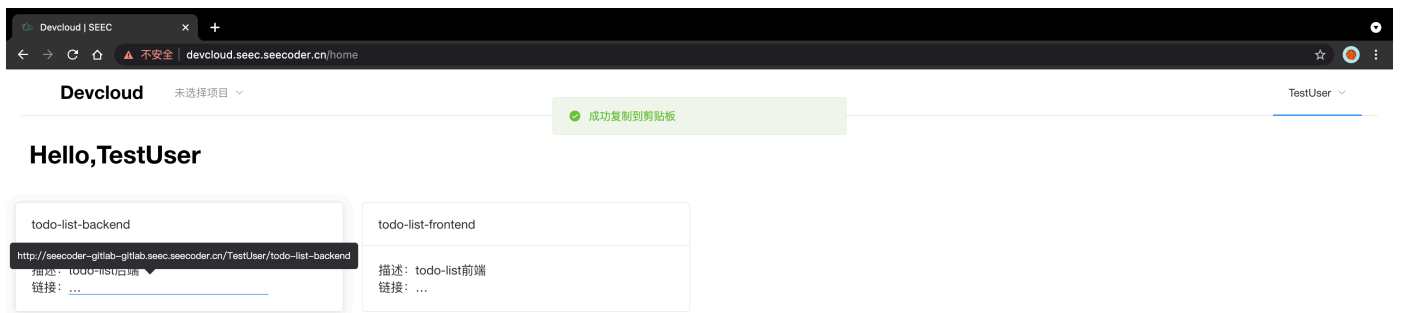
为前端、后端各创建一个项目

创建完如下图，有todo-list-backend以及todo-list-frontend一个前端、一个后端项目（链接是显示问题，请勿担心）



1.3 上传项目

使用Git，通过自动创建的仓库地址，clone项目到本地



将项目地址复制，加上.git后缀，加上注册时的账户和密码作验证信息

git clone `http://{用户名/邮箱}:{密码}@seecoder-gitlab-gitlab.seecoder.cn/deponia/course-learning-backend.git`

注意使用邮箱做验证信息的时候 邮箱中的@要进行转码为%40，后面分隔url和验证信息的@无需转码, 使用用户名则没有关系

例如：

git clone <http://321321321%40qq.com:123123123@seecoder-gitlab-gitlab.seecoder.cn/deponia/course-learning-backend.git>

git clone <http://myname:123123123@seecoder-gitlab-gitlab.seecoder.cn/deponia/course-learning-backend.git>

```
zhaoxingrui@zhaoxingruiMacBook-Pro todolist % git clone http://[redacted]@seecoder-gitlab-gitlab.seecoder.cn/TestUser/todo-list-backend.git
Cloning into 'todo-list-backend'...
warning: redirecting to https://seecoder-gitlab-gitlab.seecoder.cn/TestUser/todo-list-backend.git/
warning: You appear to have cloned an empty repository.
zhaoxingrui@zhaoxingruiMacBook-Pro todolist % git clone http://[redacted]@seecoder-gitlab-gitlab.seecoder.cn/TestUser/todo-list-frontend.git
Cloning into 'todo-list-frontend'...
warning: redirecting to https://seecoder-gitlab-gitlab.seecoder.cn/TestUser/todo-list-frontend.git/
warning: You appear to have cloned an empty repository.
zhaoxingrui@zhaoxingruiMacBook-Pro todolist %
```

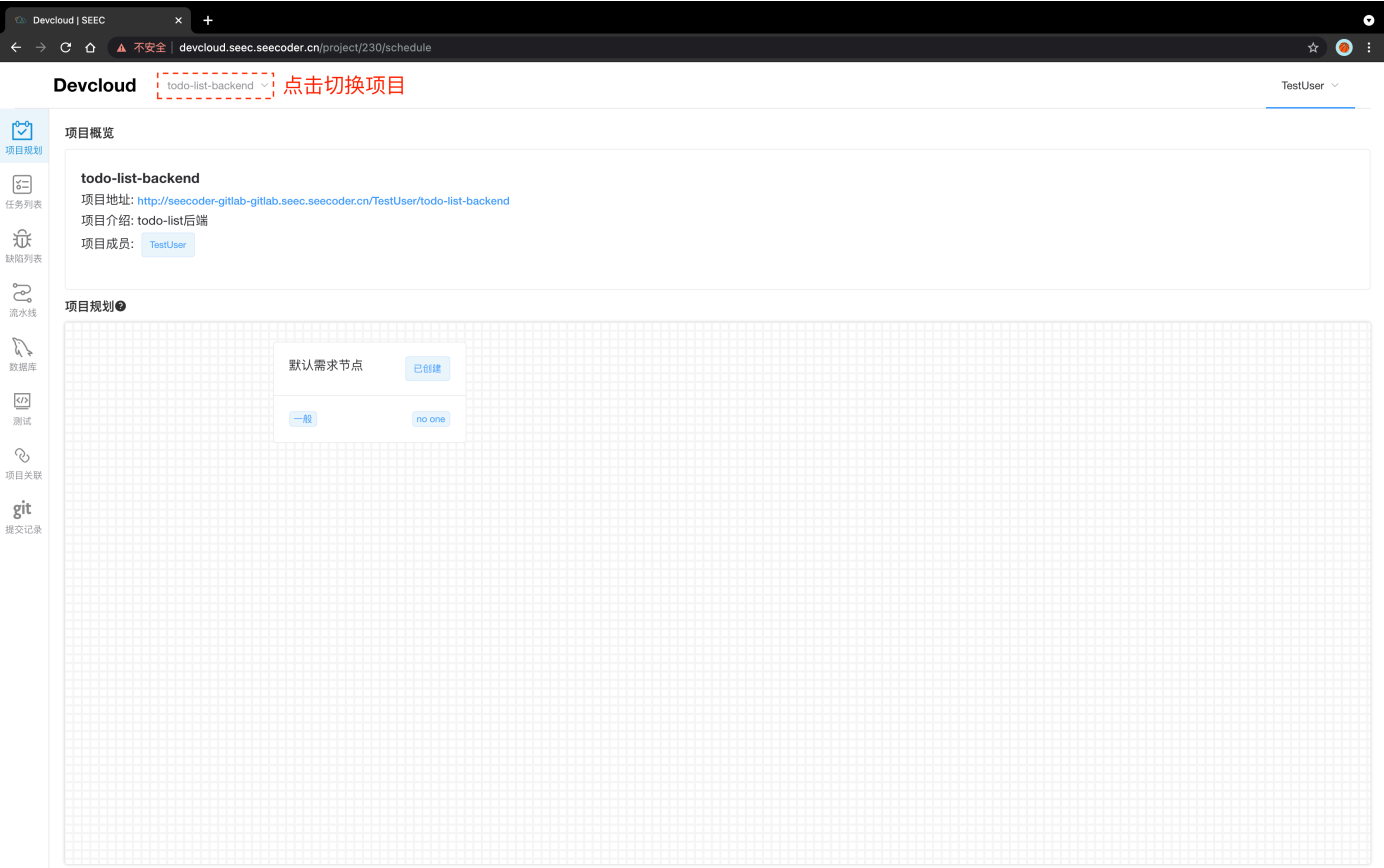
之后将前、后端复制过来，**注意不要把老的.git文件复制过来**，只复制项目文件。将项目上传至各自的仓库，在新的仓库上协作开发。

```
git add .
git commit -m "init: 初始化"
git push
```

```
zhaoxingrui@zhaoxingruiMacBook-Pro todo-list-backend % git add .
zhaoxingrui@zhaoxingruiMacBook-Pro todo-list-backend % git commit -m "init: 初始化"
[master (root-commit) 3ababc3] init: 初始化
13 files changed, 816 insertions(+)
create mode 100644 .gitignore
create mode 100644 .mvn/wrapper/MavenWrapperDownloader.java
create mode 100644 .mvn/wrapper/maven-wrapper.jar
create mode 100644 .mvn/wrapper/maven-wrapper.properties
create mode 100755 mvnw
create mode 100644 mvnw.cmd
create mode 100644 pom.xml
create mode 100644 src/main/java/com/example/demo/DemoApplication.java
create mode 100644 src/main/java/com/example/demo/controller/taskController.java
create mode 100644 src/main/java/com/example/demo/dao/TaskRepository.java
create mode 100644 src/main/java/com/example/demo/entity/task.java
create mode 100644 src/main/resources/application.yml
create mode 100644 src/test/java/com/example/demo/DemoApplicationTests.java
zhaoxingrui@zhaoxingruiMacBook-Pro todo-list-backend %
```

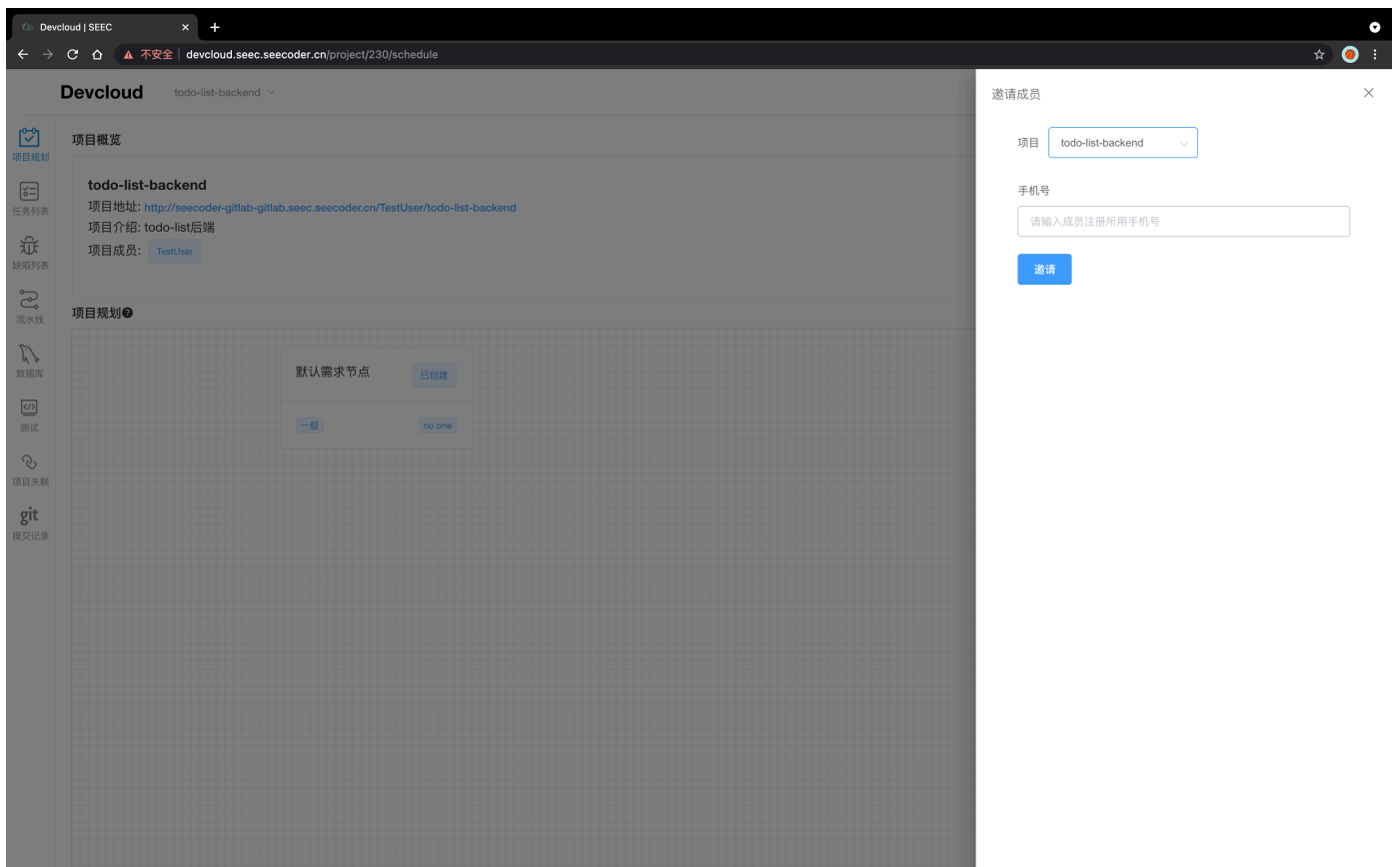
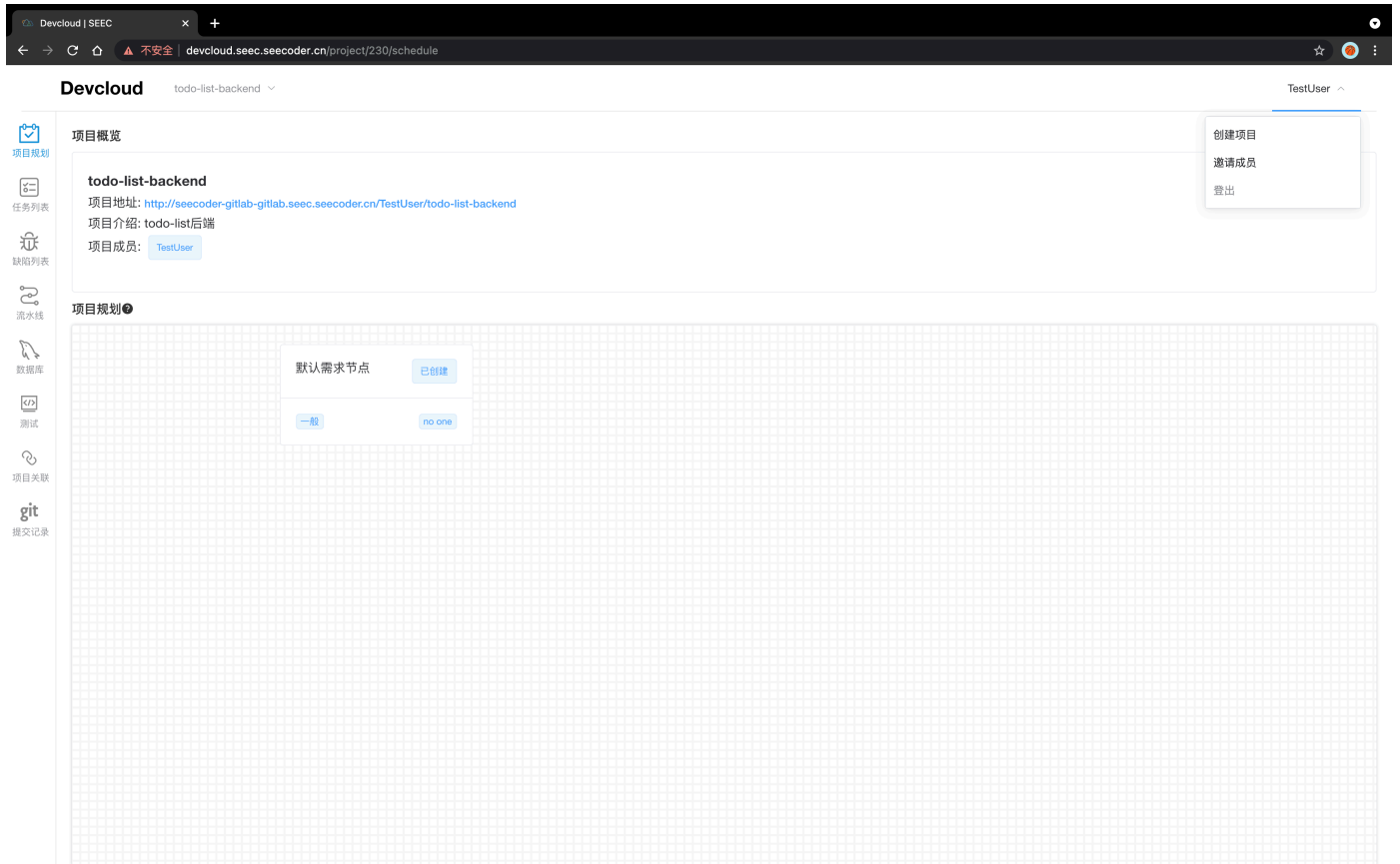
1.4 项目主页

点击项目卡片进入项目主页

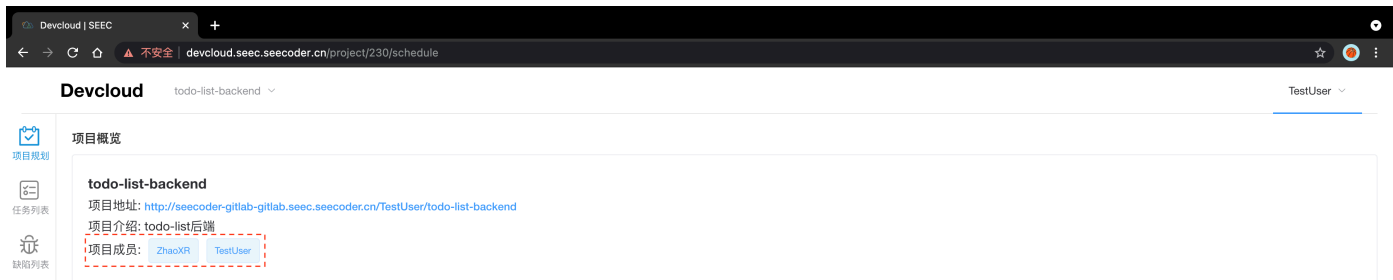


1.5 邀请成员

填写组员注册时使用的手机号即可邀请成员



邀请成员后可以发现该项目的成员已经更新了



2. 项目规划-需求

2.1 功能介绍

需求为一个深度为4的树。用户可以自由新增删除需求节点。

每一层对应一类需求，从左到右/从粗粒度到细粒度，对应着Epic、Feature、Story、Task四类需求

含义如下：[Epic & Feature & Story & Task - 编程侠Java - 博客园\(cnblogs.com\)](#)

每个需求节点可以修改名称、设置紧急度、工时、指派负责人等。

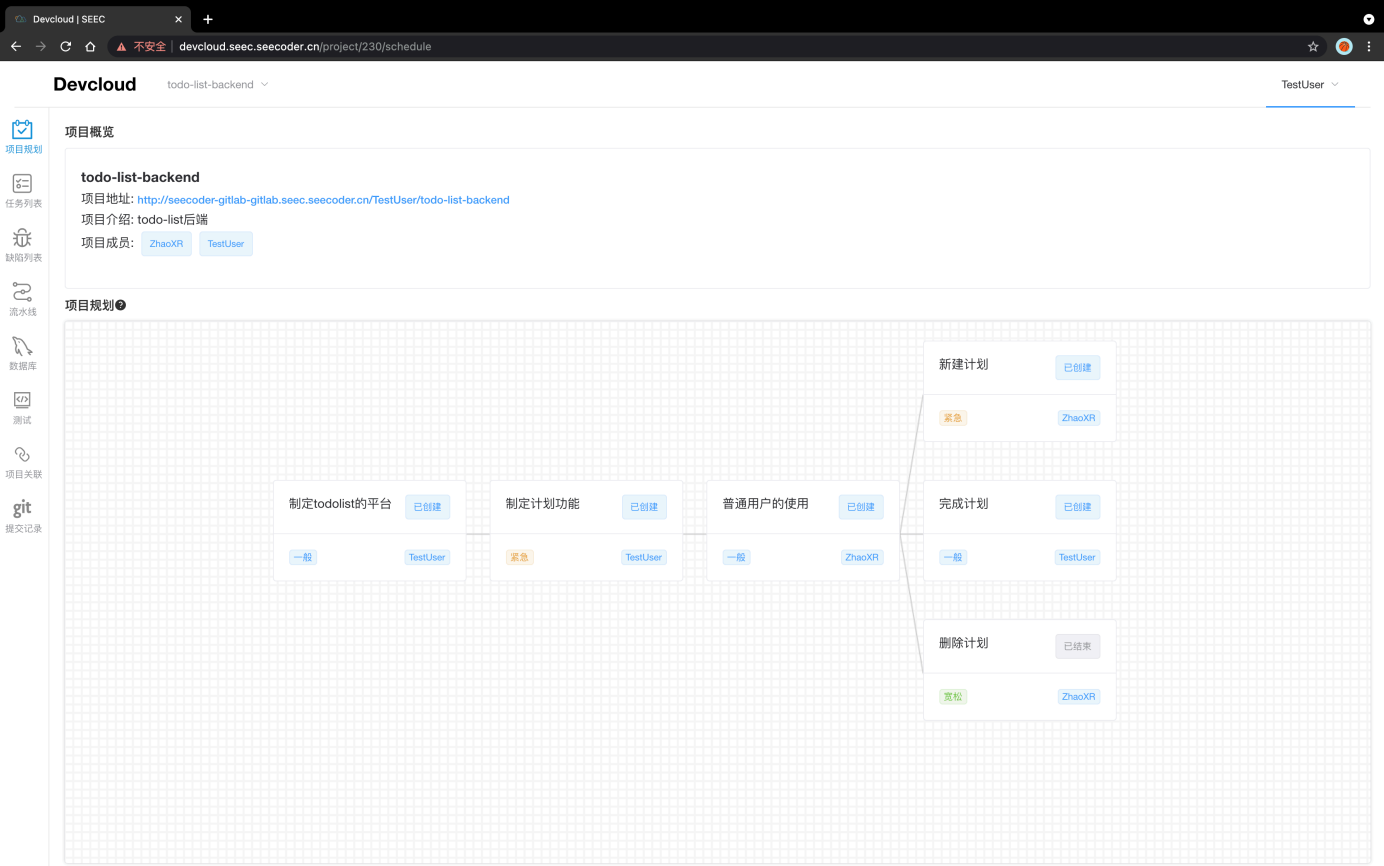
点击[项目规划](#)字段右边的问号符号即可查看需求树的4层说明及举例

需求规划四层说明



类型	说明
需求(Requirement)	- 需求是用户的一种期望，是用户为了解决问题或达到某些目标所需要的条件或能力。 - 是系统或系统部件为了满足合同、标准、规范或其他正式文档所规定的要求而需要具备的条件或能力。 - 对上述中的一个条件或一种能力的一种文档化表述。 - 这里需求的粒度较大，需要继续分解为Feature；并通过Feature继续细化为Story来完成开发和迭代。
特性(Feature)	- 特性是可以给用户带来价值的产品功能或者特性。 - 特性向上承接Feature，向下分解为Story。相比需求，Feature更加具体形象，用户更容易理解。
用户故事(Story)	- 用户故事是从用户角度对产品需求的详细描述，具有更小的粒度。 - 一般是从不同用户出发，对同一个大粒度需求的不同希望。
任务(Task)	- 在书写完用户故事后，需要将Story指派给具体成员，并分解为一个或多个任务(Task)。 - 需要填写预计工时

下面是todolist后端项目的规划需求树：



- task一定是最小的可分配任务粒度
- 需求节点可修改配置一览



导航栏第二项**任务列表**可以快速查看所有Task需求的情况，方便了解项目进度

Devcloud | SEEC

devcloud.seecoder.cn/project/230/taskList

Devcloud todo-list-backend TestUser

清除过滤器

ID	所属项目ID	优先级	名称	负责人	预期耗时(人日)	开始时间	结束时间	状态
1324	230	紧急	新建计划	ZhaoXR	1	2021/09/14 21:19:40		已创建
1325	230	一般	完成计划	TestUser	1	2021/09/14 21:19:42		已创建
1326	230	宽松	删除计划	ZhaoXR	1	2021/09/14 21:19:44	2021/09/14 21:20:28	已结束

若使用如下代码提交规范

```
feat: working on #29#, 题目表结构设计完成, dao层接口完成
feat: complete #29#, 创建题目用例完成
```

前者会将对应id的Task的状态变为正在进行

后者会将对应id的Task的状态变为已完成

且会帮你记录对应的commit信息

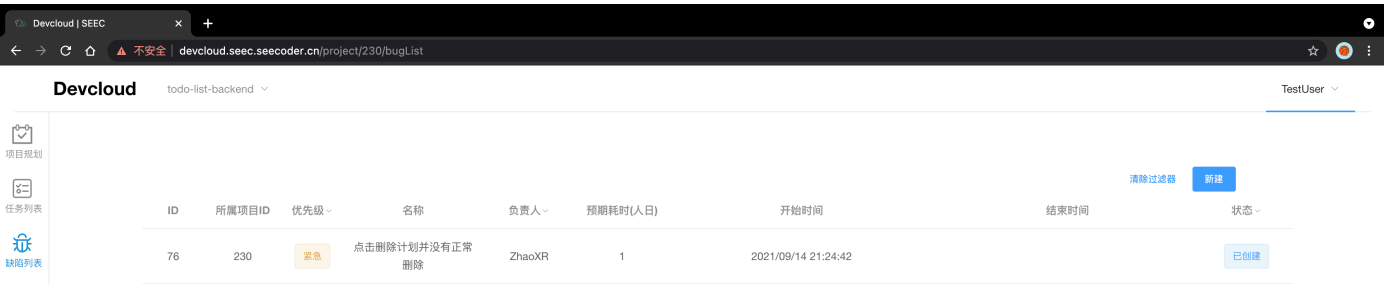
也可以直接在上述配置修改页面手动更改状态

3. bug记录功能

3.1 功能介绍

导航栏第三项**缺陷列表**可以记录项目中出现的bug

新建，可以将使用的时候遇到的Bug记录下来，设置负责人和处理时间



可修改配置一览



若使用如下代码提交规范

```
fix: fixing on #5#, 修改xxxx
fix: fix ##, xxxx已修复
```

前者会将对应id的bug的状态变为正在进行

后者会将对应id的bug的状态变为已完成

且会帮你记录对应的commit信息

也可以直接在上述配置修改页面手动更改状态

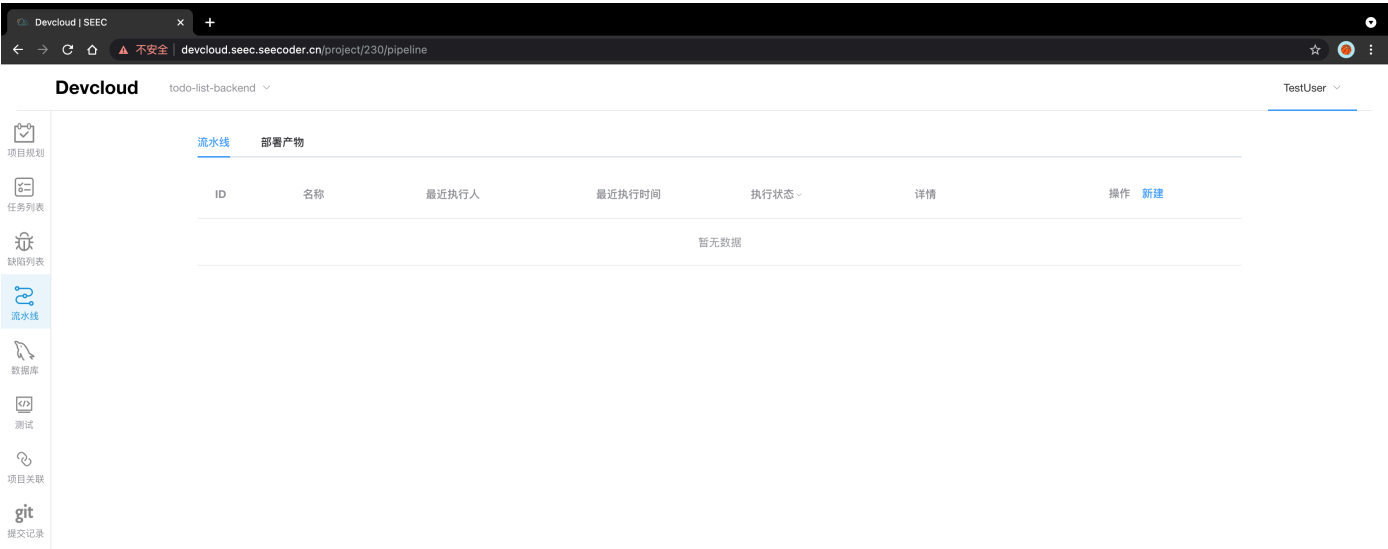
4. 部署功能

4.1 功能简介

集群资源有限，请不要部署无关的应用！

部署功能分为两个部分

- 流水线：负责部署的流程配置
- 部署产物：负责部署实例的管理



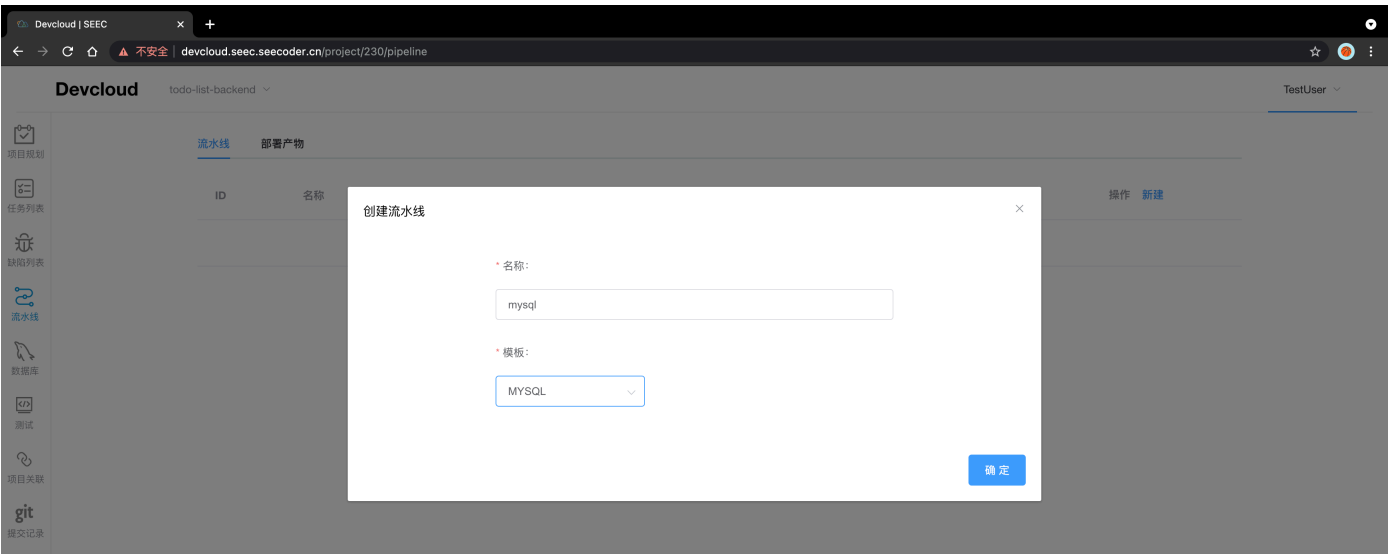
4.2 创建后端使用的Mysql

4.2.1 创建mysql

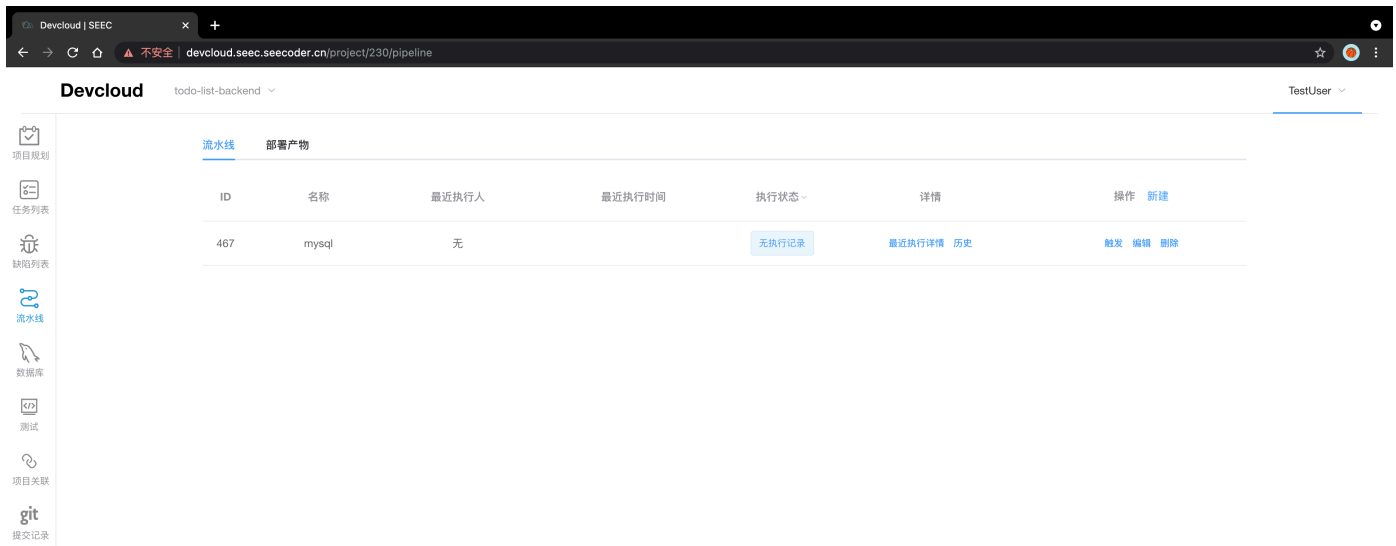
点击新建流水线

输入流水线名称，选择Mysql模板

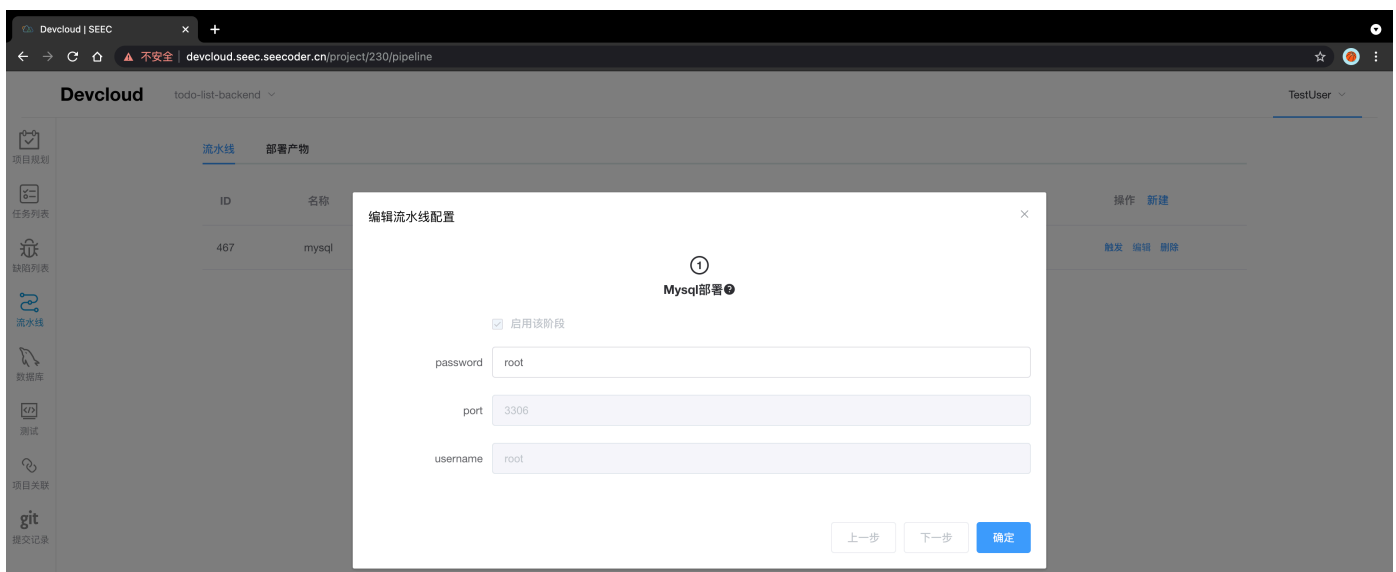
注意流水线名称不能重复, 流水线名称只能为以小写英文字母开头，且由英文字母和'-'组成的字符串，长度为3-30



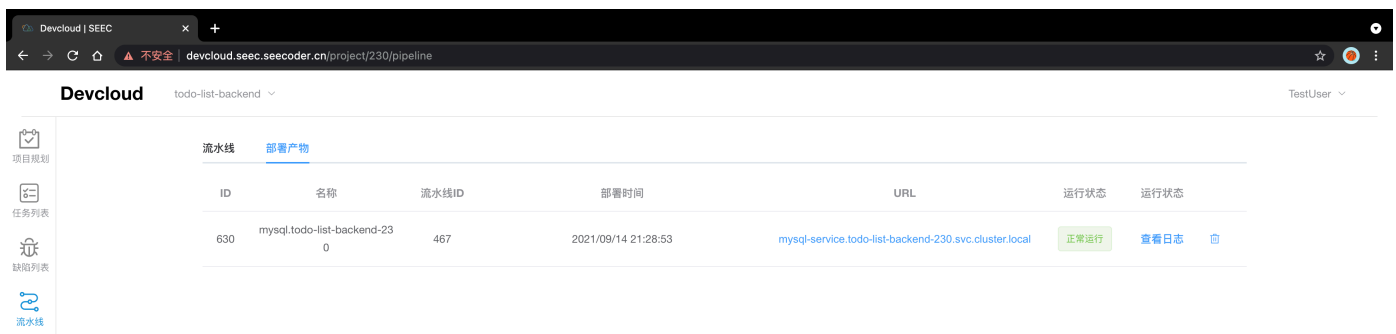
创建完成界面



点击编辑可以配置Mysql管理员密码，端口使用默认的3306



点击触发进行部署，耐心等待一段时间部署结束



4.2.2 mysql数据库、表、数据初始化

这个功能点击侧边栏第五项**数据库**

注意，这是个新加的功能，出于安全性考虑，集群无法直接暴露mysql的端口，只能用http请求的方式执行sql，线上使用体验可能有点差。所以尽可能地在本地调试好应用，防止线上查bug的时候比较麻烦

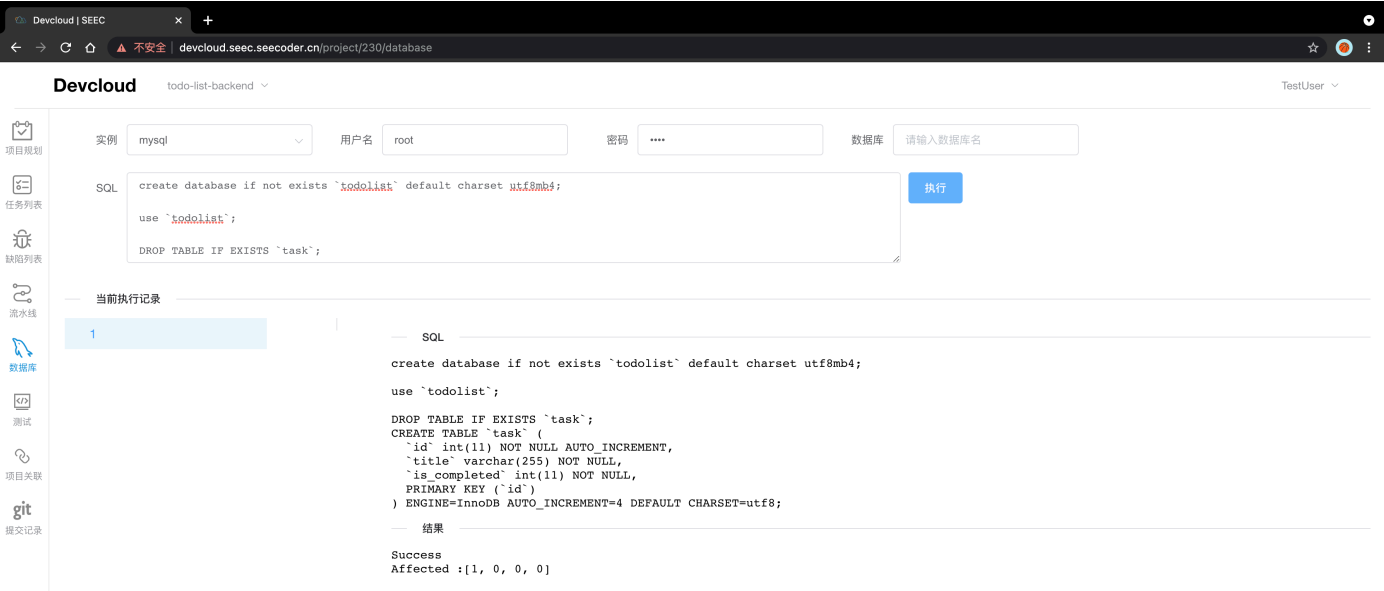
下拉选择刚刚部署的Mysql实例，填入用户名密码

数据库名称空着(因为你还没建)

将建表sql语句复制到下面执行

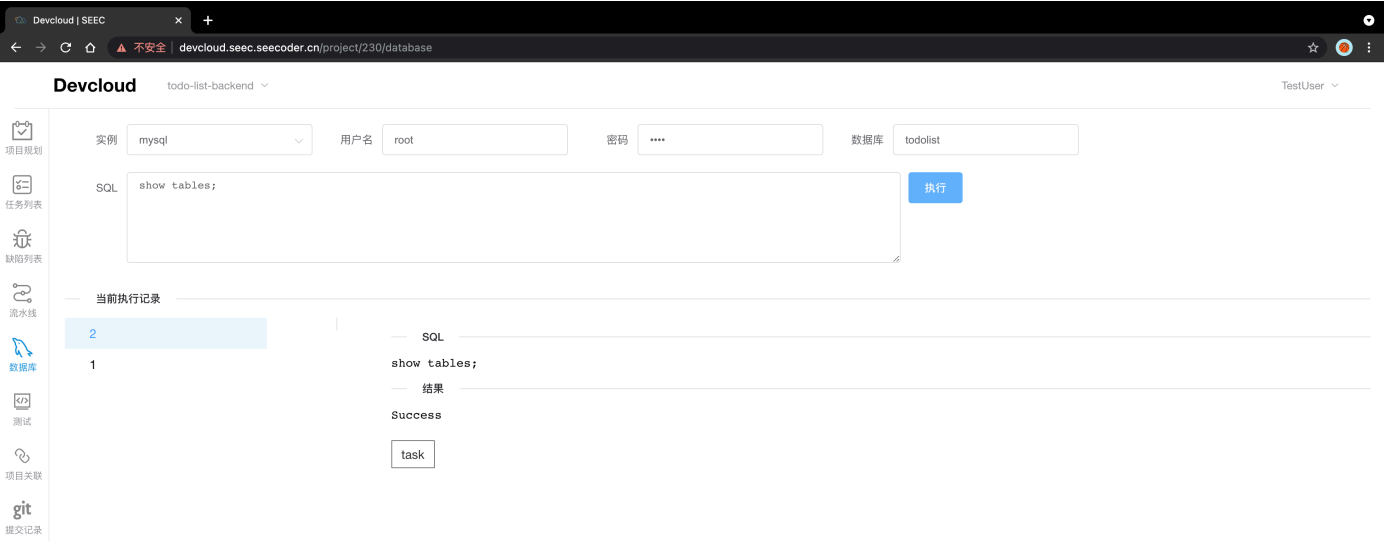
执行结果如下，数字代表每一句sql的影响（新增/删除）行数

有些为0，是因为有些sql本来没有新增或删除

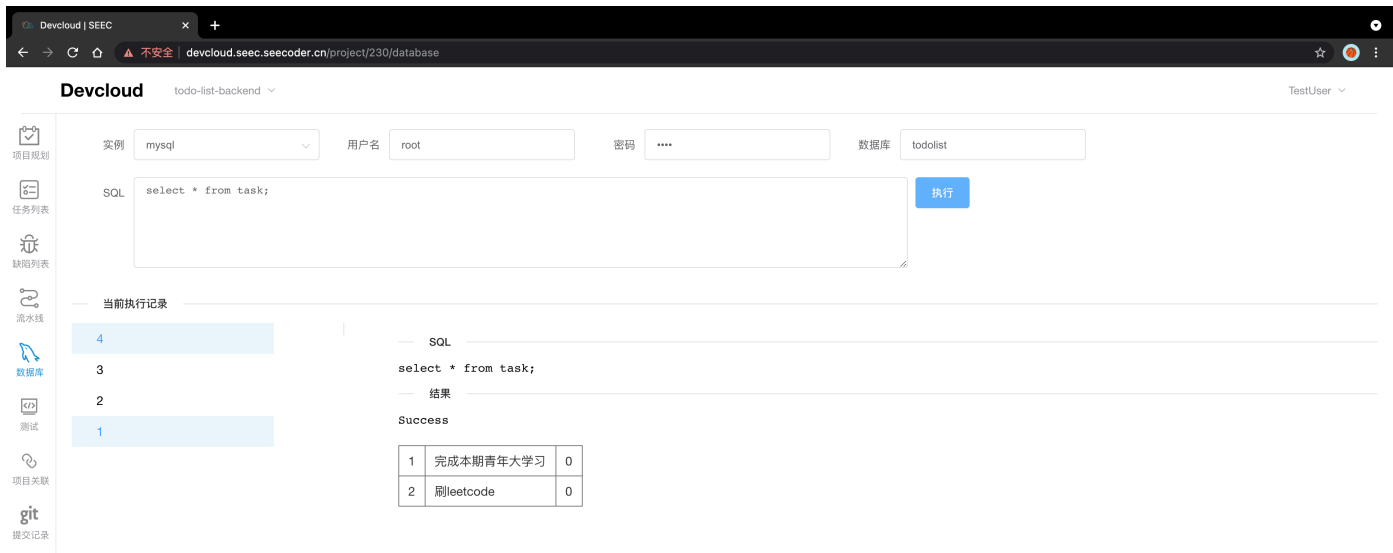


初始化完成后，你就可以填入你的数据库名称来查询了

示例：填入我们刚创建的数据库todolist, 执行一下show tables， 能看见我们刚刚创建的表 task



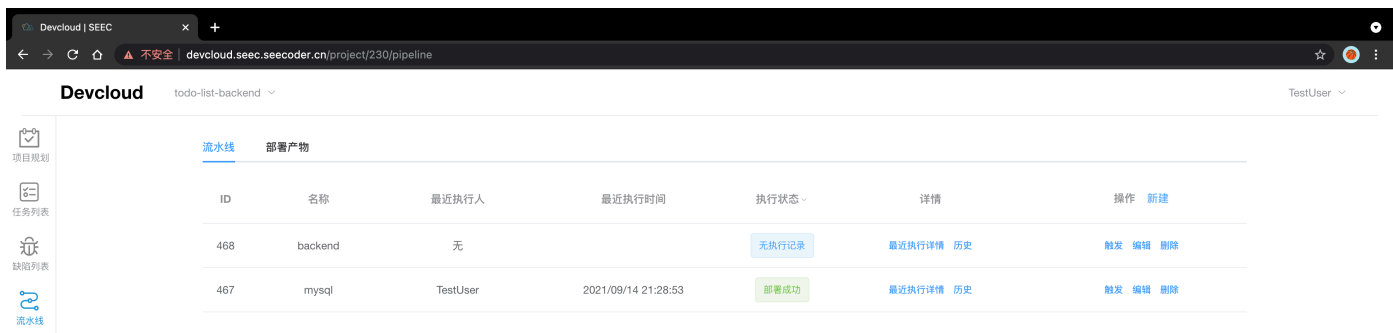
我们可以插入两条数据，插入后再查询的结果如下：



4.3 创建SpringBoot后端实例

点击新建流水线

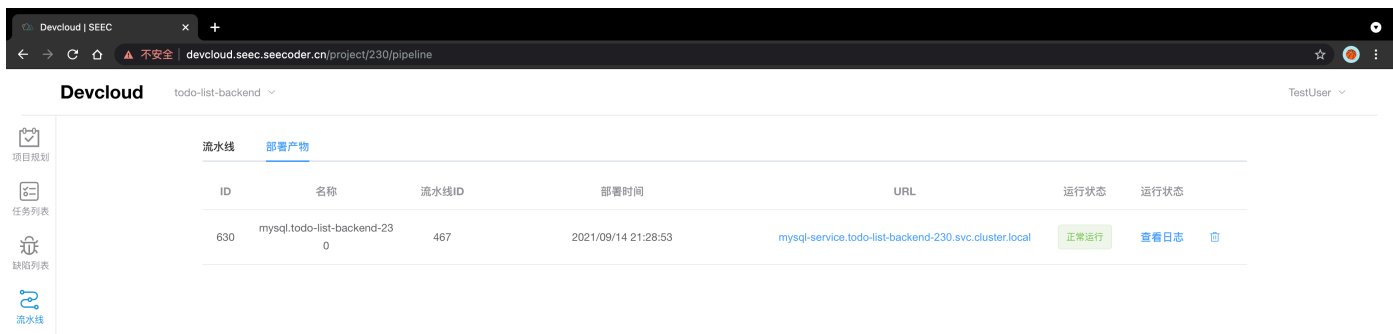
输入流水线名称，选择SPRINGBOOT_JAVA8模板（注意上述提到的流水线名称规范）



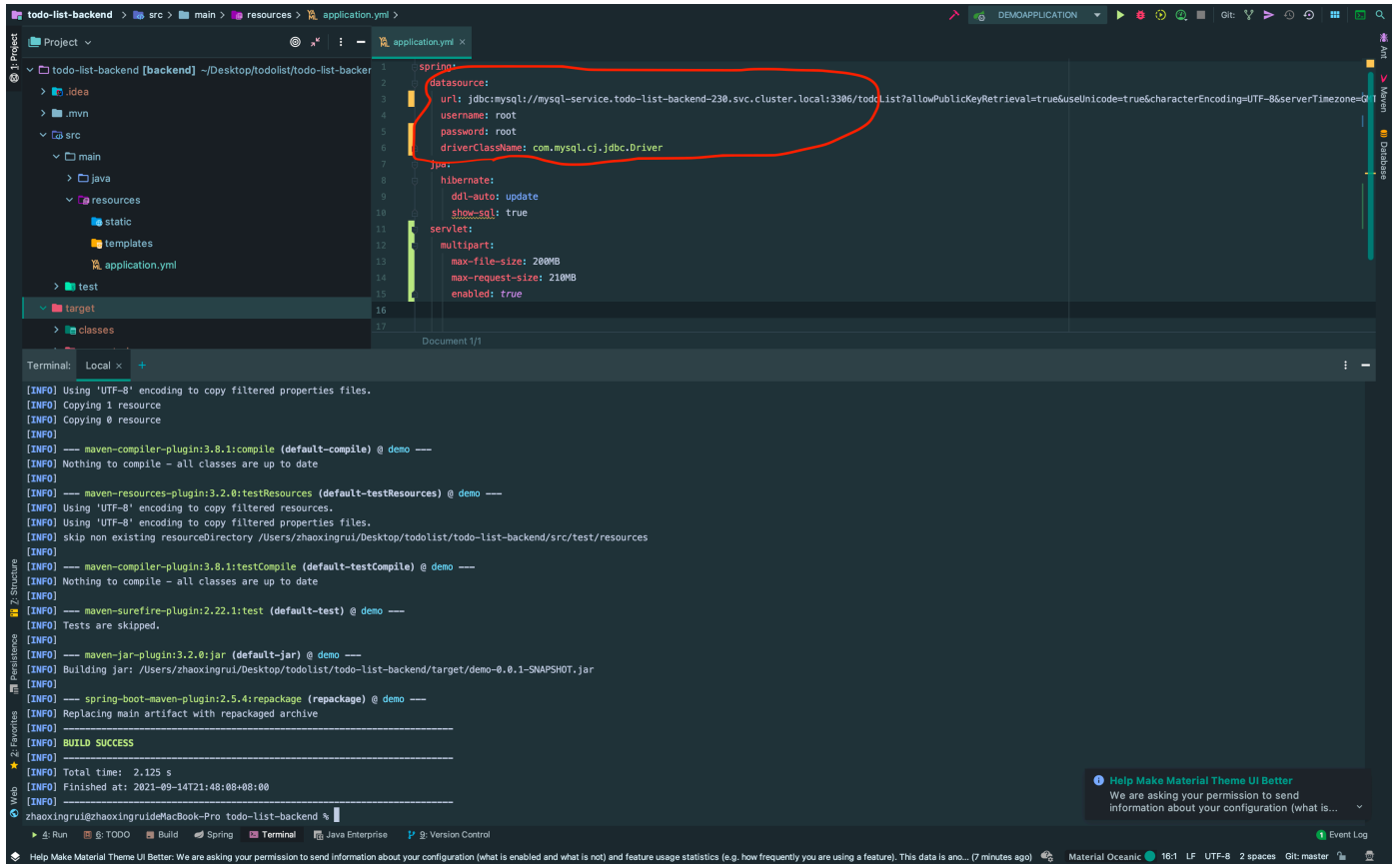
4.3.1 代码配置

1. 将后端连接的mysql配置为与线上的配置一致

- url: 下图mysql部署产物的url
- 你使用的数据库名
- 管理员密码

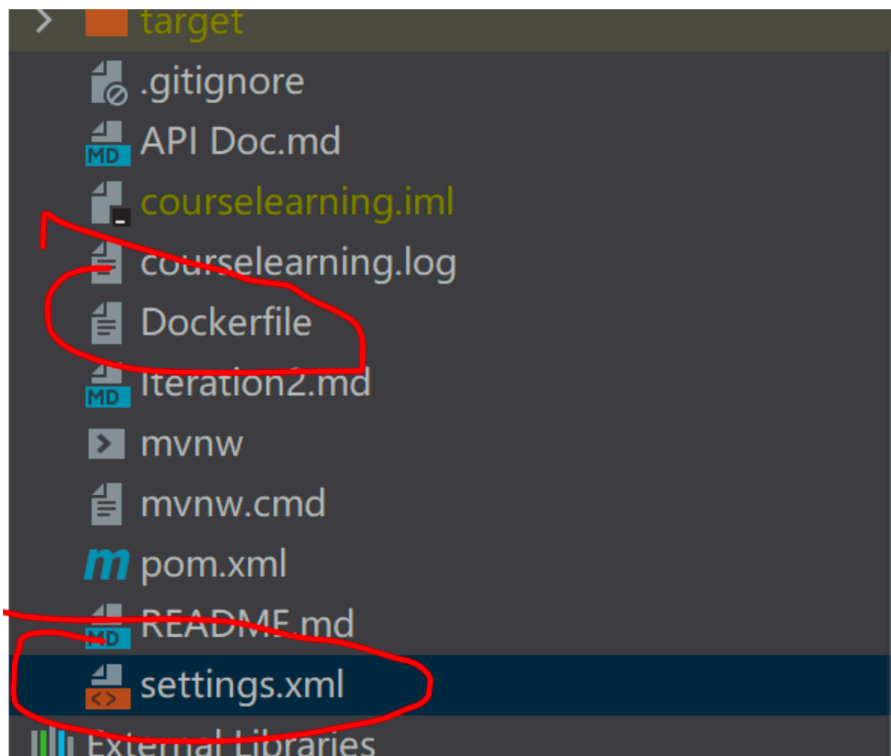


按以上设置修改后端的application.yml



2. 删除根目录下的Dockerfile文件和Settings.xml文件

这两个文件将由平台提供，不删除会导致报错

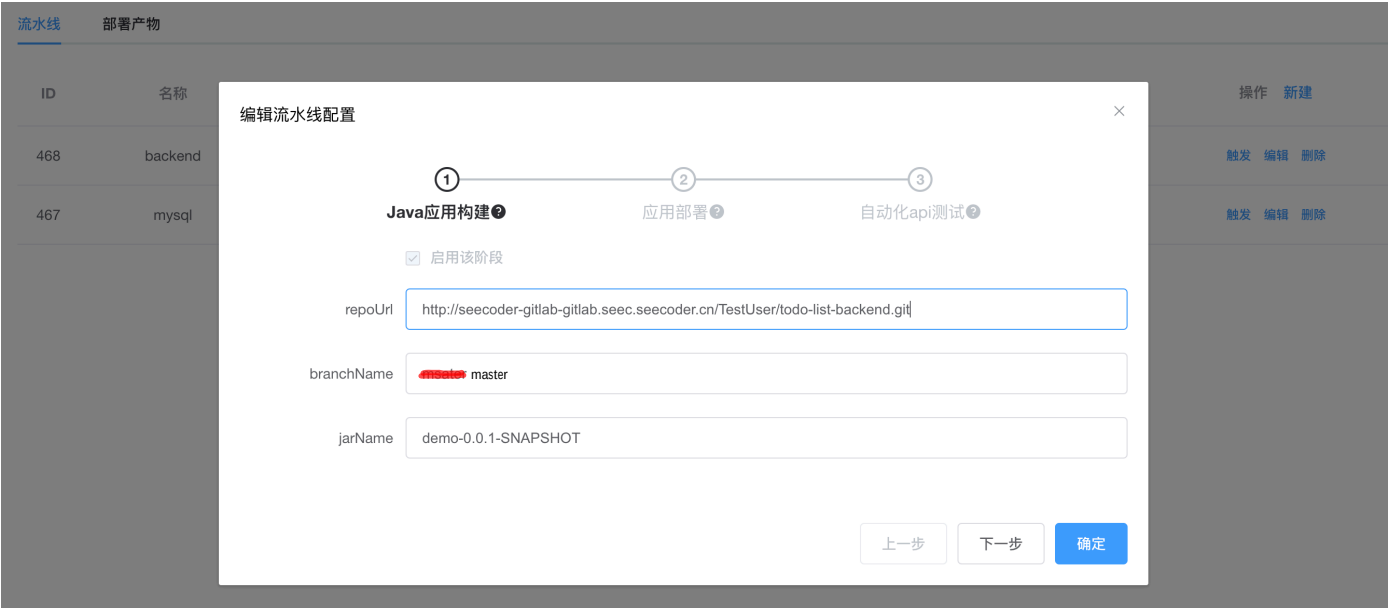


3. 更改完之后记得push变更

4.3.2 流水线配置

第一页配置

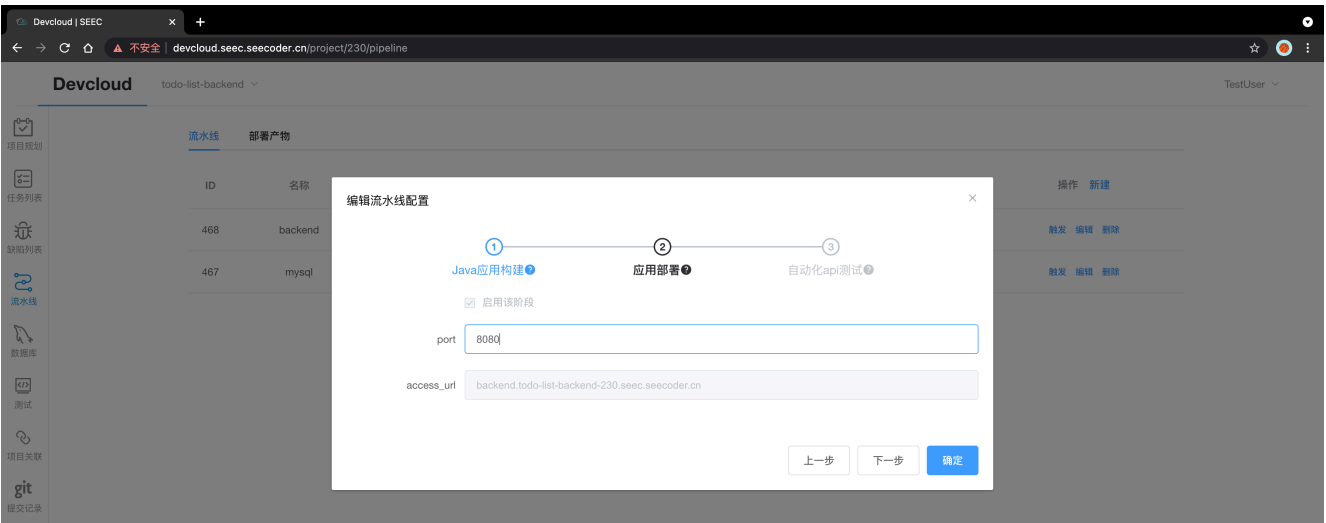
- 1. repoUrl：项目地址url用于clone，不需要上文的验证信息，请在结尾加上.git
如：<http://seecoder-gitlab-gitlab.seec.seecoder.cn/TestUser/todo-list-backend.git>
 - 2. branchName：需要用于部署的分支，请确保这条分支的应用可以正确运行
 - 3. jarName: 务必要和你打包出来的jar包名字一致，不需要.jar后缀
- 如：打包出来的应用叫demo-0.0.1-SNAPSHOT.jar，那么填入的值为demo-0.0.1-SNAPSHOT



不知道打包名称的，使用ide的maven package打包一次，到 项目根目录/target 下查看jar包名称

第二页配置

- 1. port：务必和你后端springboot application.yml配置暴露的端口一致（默认是8080）



第三页配置

将启用该阶段勾除，暂不使用该阶段（有接口测试时可以启用）

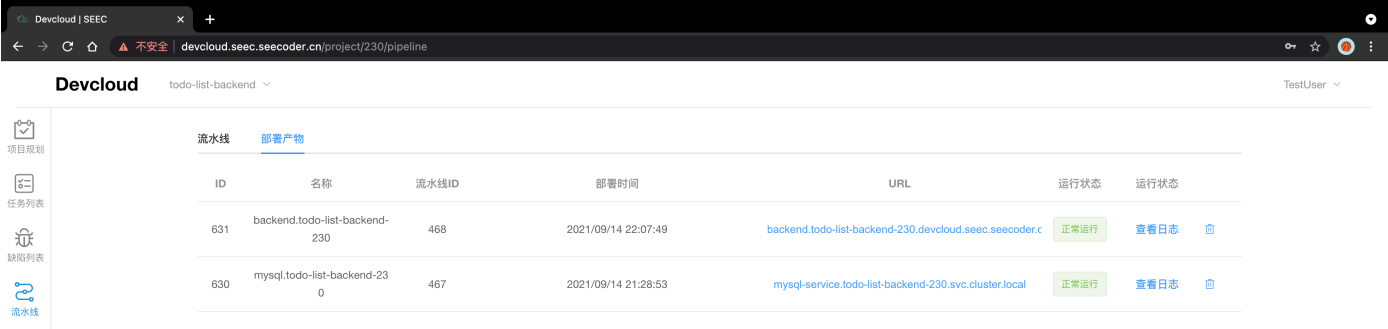
4.3.3 部署后端实例

点击触发进行部署

部署过程大概持续5-10min，请耐心等待

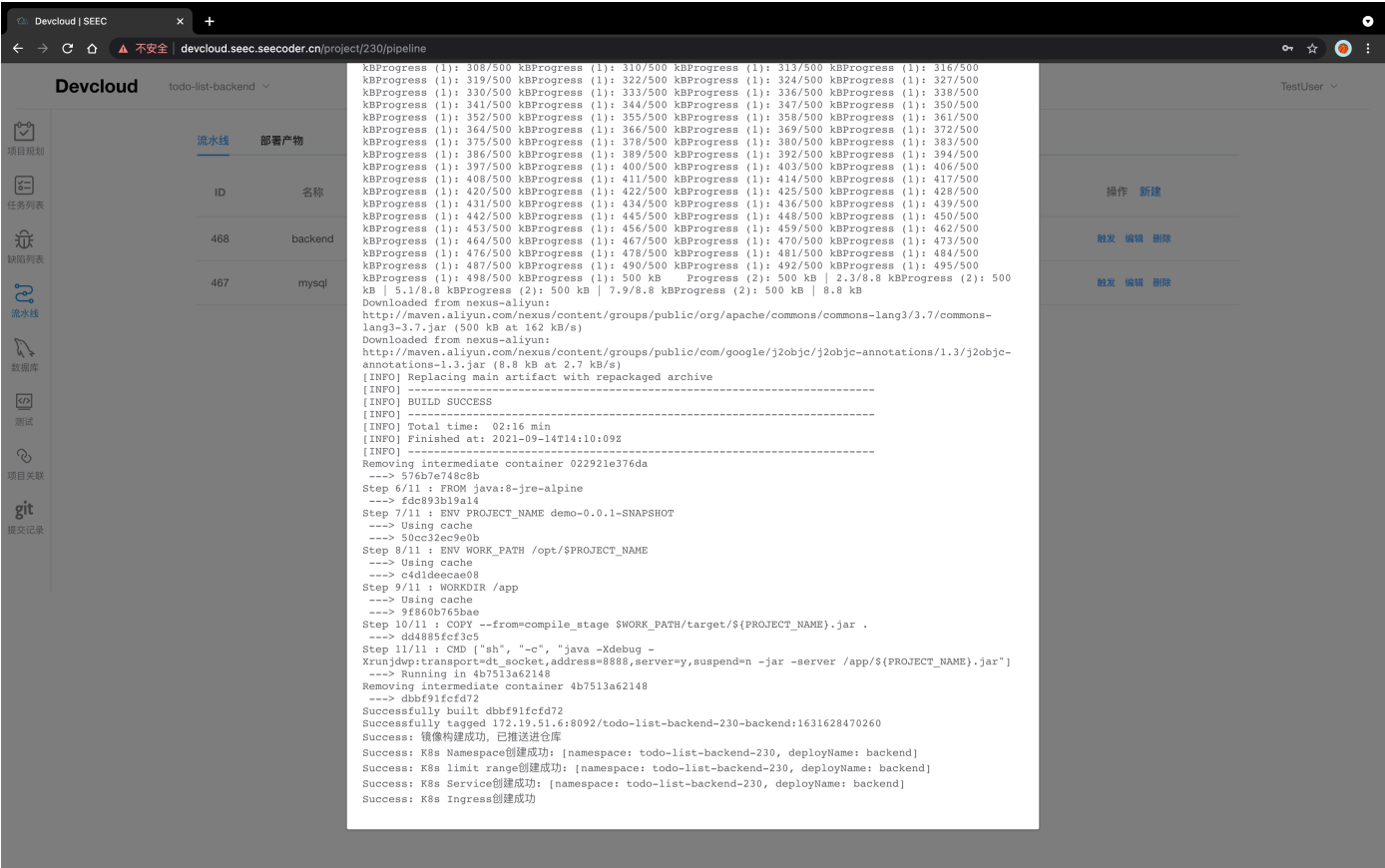
部署成功之前，部署产物会显示不存在（因为读取实例状态未读取到），请不要惊慌

部署成功



部署成功之后点击最后执行详情可以看见构建部署过程

部署失败请到也请到这里看部署日志，如果出现不能解决的问题联系我们



部署实例产物界面，并可以查看实例日志

线上Debug主要在这个日志查询问题

Devcloud SEEC							
devcloud.seec.seecoder.cn/project/230/pipeline							
Devcloud todo-list-backend TestUser							
项目规划 任务列表 缺陷列表 流水线	流水线		部署产物				
	ID	名称	流水线ID	部署时间	URL	运行状态	运行状态
	631	backend.todo-list-backend-230	468	2021/09/14 22:07:49	backend.todo-list-backend-230.devcloud.seec.seecoder.cn	正常运行	查看日志 🗑
	630	mysql.todo-list-backend-230	467	2021/09/14 21:28:53	mysql-service.todo-list-backend-230.svc.cluster.local	正常运行	查看日志 🗑

Devcloud SEEC							
devcloud.seec.seecoder.cn/project/230/pipeline							
Devcloud todo-list-backend TestUser							
项目规划 任务列表 缺陷列表 流水线 数据库 测试 项目关联 git 历史记录	流水线		部署产物				
	ID	名称	流水线ID	部署时间	URL	运行状态	运行状态
	631	back					查看日志 🗑
	630	mys					查看日志 🗑

产物日志

Listening for transport dt_socket at address: 8888



```
2021-09-14 14:11:14.543 INFO 6 --- [main] com.example.demo.DemoApplication : Starting DemoApplication
v0.0.1-SNAPSHOT using Java 1.8.0.111-internal on backend-deployment-79b647997d-5zblv with PID 6 (/app/demo-0.0.1-SNAPSHOT.jar started by root in /app)
2021-09-14 14:11:14.742 INFO 6 --- [main] com.example.demo.DemoApplication : No active profile set,
falling back to default profiles: default
2021-09-14 14:12:02.341 INFO 6 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Bootstrapping Spring Data
JPA repositories in DEFAULT mode.
2021-09-14 14:12:05.144 INFO 6 --- [main] .s.d.r.c.RepositoryConfigurationDelegate : Finished Spring Data
repository scanning in 2696 ms. Found 1 JPA repository interfaces.
2021-09-14 14:12:38.442 INFO 6 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with
port(s): 8080 (http)
2021-09-14 14:12:38.941 INFO 6 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2021-09-14 14:12:38.942 INFO 6 --- [main] org.apache.catalina.core.StandardEngine : Starting Servlet engine:
[Apache Tomcat/9.0.52]
2021-09-14 14:12:40.945 INFO 6 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring
embedded WebApplicationContext
2021-09-14 14:12:40.945 INFO 6 --- [main] w.s.c.ServletWebServerApplicationContext : Root
WebApplicationContext: initialization completed in 82002 ms
2021-09-14 14:13:04.741 INFO 6 --- [main] o.hibernate.jpa.internal.util.LogHelper : HHH000204: Processing
PersistenceUnitInfo [name: default]
2021-09-14 14:13:13.042 INFO 6 --- [main] org.hibernate.Version : HHH000412: Hibernate ORM
core version 5.4.32.Final
2021-09-14 14:13:30.443 INFO 6 --- [main] o.hibernate.annotations.common.Version : HCANN000001: Hibernate
Commons Annotations {5.1.2.Final}
2021-09-14 14:13:39.942 INFO 6 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Starting...
2021-09-14 14:13:50.243 INFO 6 --- [main] com.zaxxer.hikari.HikariDataSource : HikariPool-1 - Start
completed.
2021-09-14 14:13:51.943 INFO 6 --- [main] org.hibernate.dialect.Dialect : HHH000400: Using dialect:
org.hibernate.dialect.MySQL57Dialect
2021-09-14 14:14:09.945 INFO 6 --- [main] o.h.e.t.j.p.i.JtaPlatformInitiator : HHH000490: Using
JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
2021-09-14 14:14:10.143 INFO 6 --- [main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA
EntityManagerFactory for persistence unit 'default'
2021-09-14 14:14:40.443 WARN 6 --- [main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is
enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure
spring.jpa.open-in-view to disable this warning
2021-09-14 14:15:15.841 INFO 6 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s):
8080 (http) with context path ''
2021-09-14 14:15:16.744 INFO 6 --- [main] com.example.demo.DemoApplication : Started DemoApplication in
261.401 seconds (JVM running for 288.401)
2021-09-14 14:20:04.943 INFO 6 --- [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring
DispatcherServlet 'dispatcherServlet'
2021-09-14 14:20:04.943 INFO 6 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet
'dispatcherServlet'
2021-09-14 14:20:04.945 INFO 6 --- [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization
in 2 ms
2021-09-14 14:21:30.741 WARN 6 --- [nio-8080-exec-4] .w.s.m.s.DefaultHandlerExceptionResolver : Resolved
[org.springframework.web.HttpRequestMethodNotSupportedException: Request method 'POST' not supported]
```

4.3.4 测试

部署好之后，确认运行日志无异常之后，可以尝试调用接口，确认后端是否正确运行。

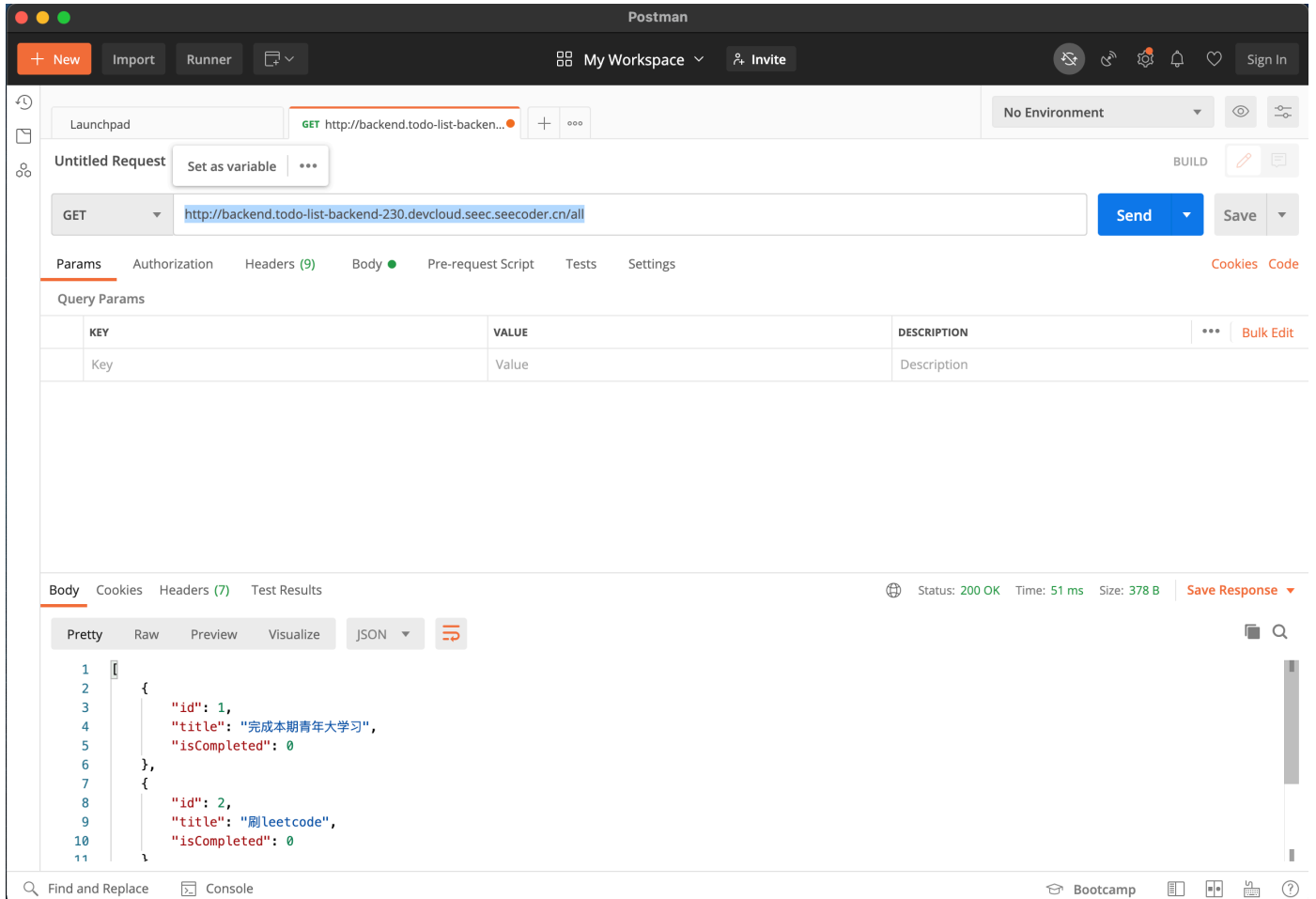
到部署界面查看 url

比如查到的是 backend.todo-list-backend-230.devcloud.seec.seecoder.cn

就通过这个地址使用http请求接口，**注意请求url不用写端口号**

使用curl、postman之类的工具进行接口测试

<http://backend.todo-list-backend-230.devcloud.seec.seecoder.cn/all>



成功

如果使用postman测试接口出现502报错，请耐心等待一会再进行测试

4.4 创建Vue.js前端实例

切换到前端项目

点击新建流水线

输入流水线名称，选择NODE_10模板

4.4.1 流水线配置

第一页配置

1. repoUrl: 项目地址url用于clone，**不需要用户、密码验证信息，请在结尾加上.git**
例如: <http://seecoder-gitlab-gitlab.seec.seecoder.cn/TestUser/todo-list-frontend.git>
2. branchName: 需要用于部署的分支，请确保这条分支的应用可以正确运行
3. nginxConfig: **使用下面的配置，注意填入的后端url需要稍作修改**
 - 后端url修改规则如下
我们在后端部署产物的界面，看见的后端url为这种形式
`backend.todo-list-backend-230.devcloud.seec.seecoder.cn`

改为这种形式

backend-service.todo-list-backend-230.svc.cluster.local

也就是在第一个分隔'.'前加入-service, 将后缀替换为.svc.cluster.local 将其填入配置

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    root /var/www/html/dist;
    location ^~/api/ {
        proxy_pass http://{替换后端url}:{后端暴露的端口}/;
        proxy_redirect http://{替换后端url}:{后端暴露的端口}/ /;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header X-Forwarded-Port $server_port;
    }
    location ~ / {
        try_files $uri $uri/ /index.html;
    }
}
```

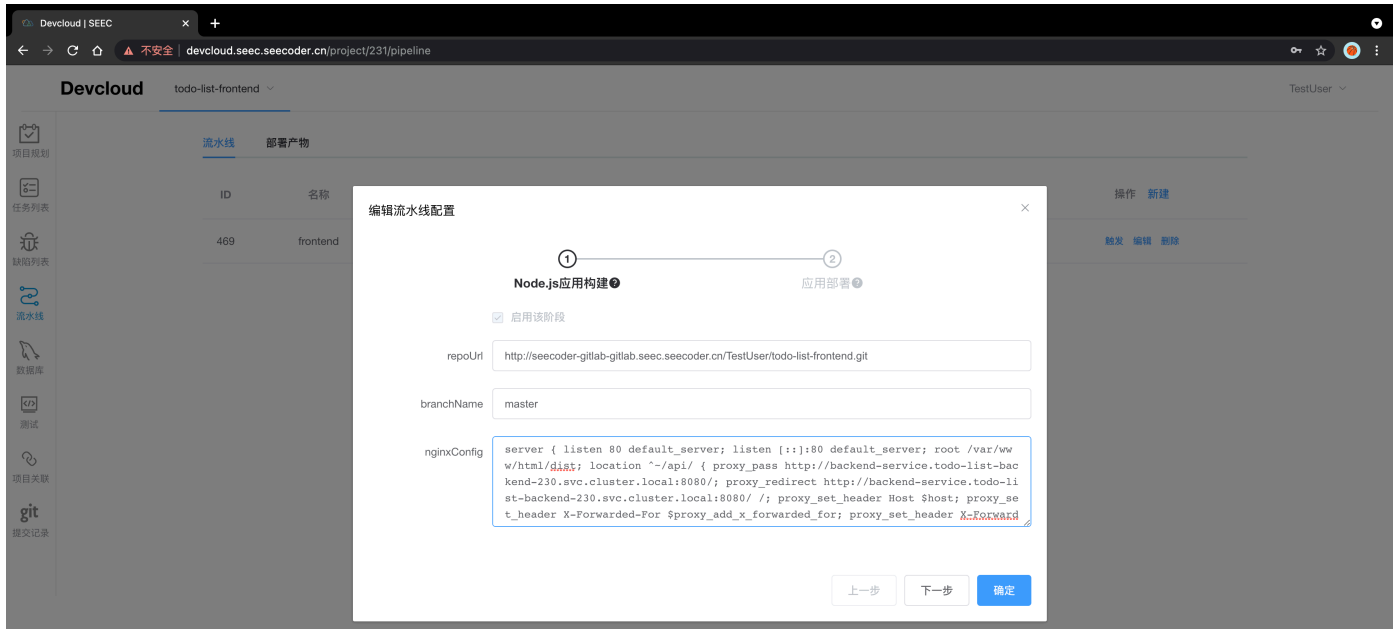
*.devcloud.seec.seecoder.cn 用于外部访问集群服务，也就是上文可以使用postman对后端进行接口调用的原因

*.svc.cluster.local 用于k8s集群内部访问自己的服务，故这里要改为这种形式，用于集群内部前端将请求转发给后端

这里仅作了解，无需深入

如果后端接口添加了/api， proxy_pass和proxy_redirect最后的 / 就不需要添加；反之则需要添加

1	若访问 http://nginx_server/test/index.html
2	
3	1,若 location /test/ {
4	proxy_pass http://192.168.1.1/ #有斜杠
5	}
6	则该请求被代理到 http://192.168.1.1/index.html
7	
8	2,若 location /test/ {
9	proxy_pass http://192.168.1.1 #没有斜杠
10	}
11	则该请求被代理到 http://192.168.1.1/test/index.html



第二页配置

1. port: 请设置为80

4.4.2 部署前端实例

点击触发进行部署

部署过程大概持续5-10min，请耐心等待

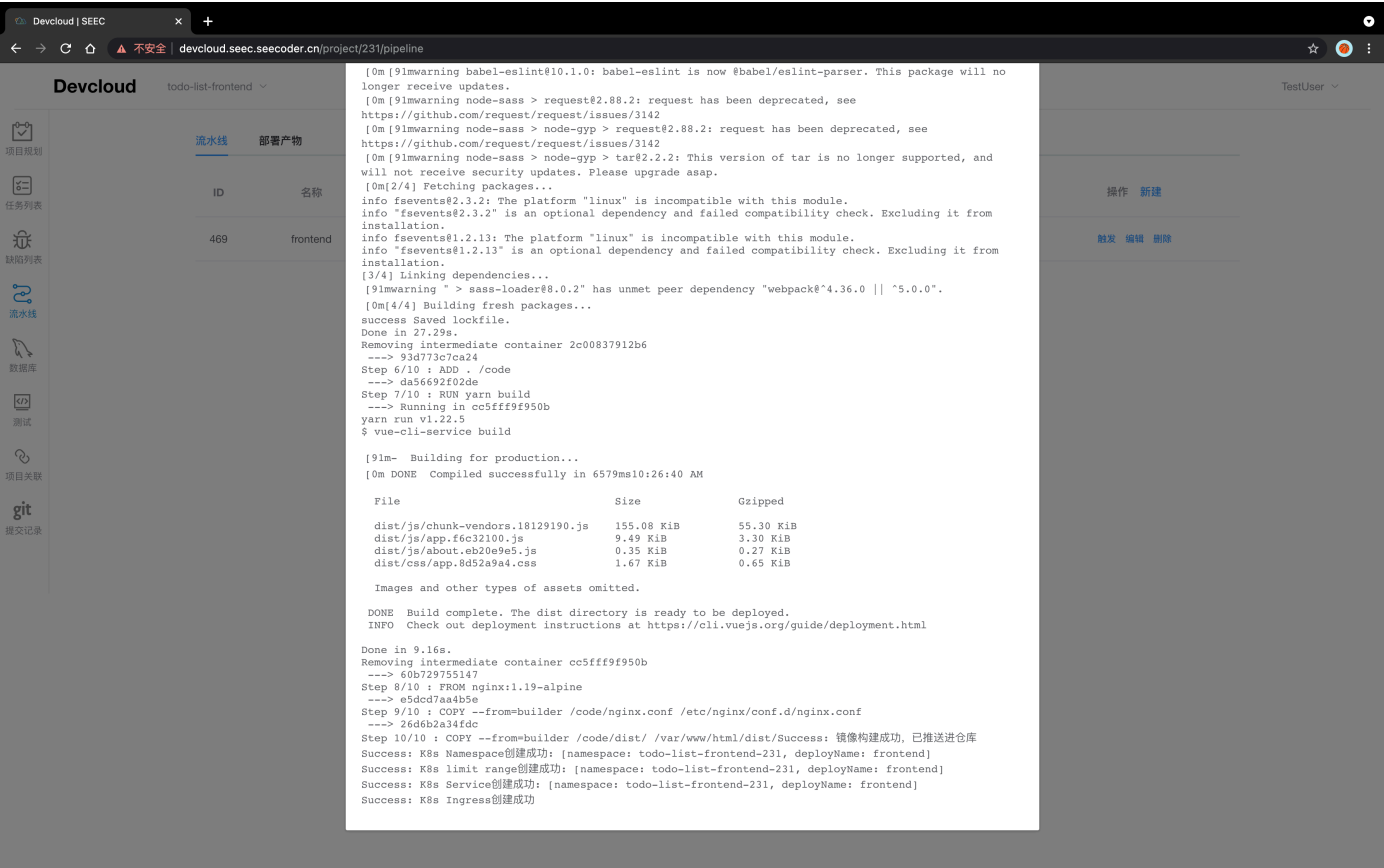
部署成功之前，部署产物会显示不存在（因为读取实例状态未读取到），请不要惊慌

部署成功

流水线							部署产物
ID	名称	最近执行人	最近执行时间	执行状态	详情	操作	新建
469	frontend	TestUser	2021/09/15 18:12:03	部署成功	最近执行详情 历史	触发 编辑 删除	

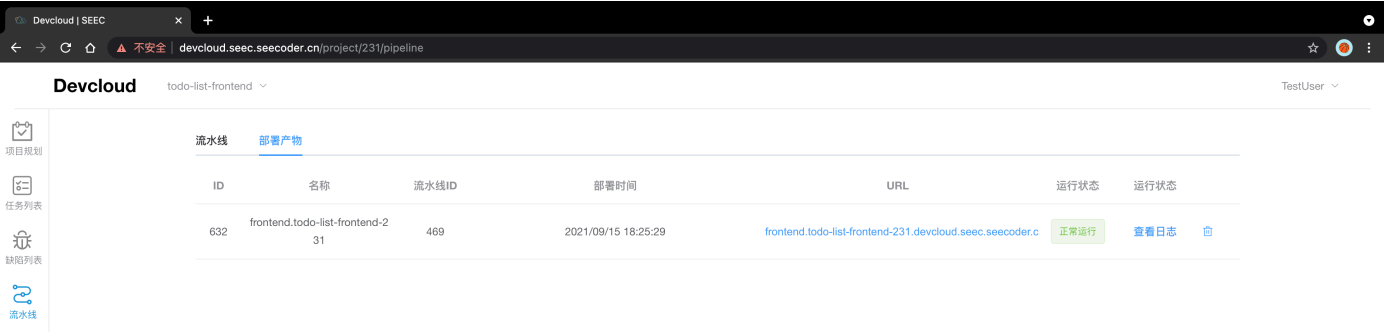
部署成功之后点击最后执行详情可以看见构建部署过程

部署失败请到也请到这里看部署日志，如果出现不能解决的问题联系我们



部署实例产物界面，并可以查看实例日志

前端线上Debug主要在浏览器Debug，这个实例日志没什么用的





4.4.3 测试

直接在浏览器输入前端实例的url

<http://frontend.todo-list-frontend-231.devcloud.seec.seecoder.cn>



成功

5. 测试功能

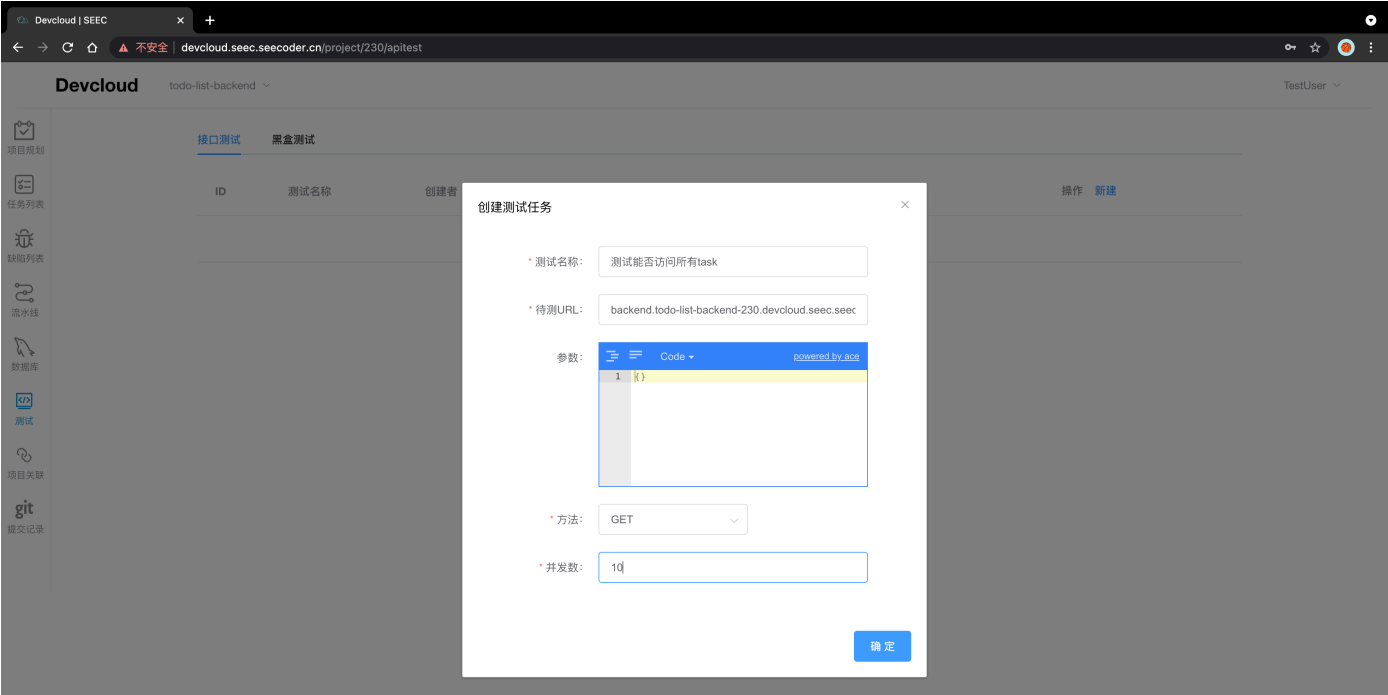
5.1 功能介绍

接口测试：为项目后端接口进行并发性能测试

黑盒测试：为项目后端的一些接口、函数、方法进行人工黑盒测试

5.2 接口测试

点击新建，填写测试任务表单，包括测试名称、后端待测URL、URL可能携带的参数、方法和并发数（10的倍数），点击确定即可添加一条接口测试任务



新建测试任务后，点击触发，即可执行接口测试任务，等待提示消息执行成功后，点击测试任务详情，可以查看该测试任务信息、执行历史及自动化测试结果（这个需要在后端流水线部署启用第三个环节，然后选择要执行的接口测试）

测试

#129

测试能否访问所有task

×

基本信息

执行历史

自动化测试结果

url:

backend.todo-list-backend-230.devcloud.seec.seecoder.cn/all

params:

Code

1 { }

method:

GET

qps:

10

ID

名称

468

backend

467

mysql

编辑流水线配置

1

Java应用构建

2

应用部署

3

自动化api测试

☒ 启用该阶段

apiTestIds

请选择

测试能否访问所有task

上一步

下一步

确定

操作

新建

触发

编辑

删除

触发

编辑

删除

点击最近一次执行结果，可以查看最近一次执行结果的信息



5.3 黑盒测试

手动验证后端接口的预期输出和实际输出对比

新建测试用例后（关联到具体的某个task），测试任务的状态默认为进行中

创建测试用例

×

title:

能否正常新建计划

taskId:

请选择

^

143

144

145

确定

点击测试用例步骤列表，可以查看属于该测试用例的步骤列表，点击新建测试用例步骤后生成一个空的步骤，测试用例步骤有描述、预期输出和状态三个属性，可以更新步骤的描述和预期输出以及更新状态

修改测试用例步骤

id:

11

description:

验证输入后按enter能否正常增加计划

expectation:

可以正常增加计划

确定

所有测试用例步骤通过后，可以点击完成测试，将测试用例的状态修改为已完成（对测试用例有疑问，也可以重新开放测试用例）

接口测试

黑盒测试

ID	标题	关联的任务ID	执行状态	详情	操作
19	能否正常新建计划	143	已完成	测试用例步骤列表	完成测试 重新开放测试 编辑 删除

点击部署的历史记录，可以查看最近一次部署的测试用例的操作记录信息

项目

#-1

×

ID	部署时间	操作类型	流水线ID
12	2021-05-14 21:16:08	finish	1

6. 项目关联功能

点击侧边栏第七项**项目关联**

可以生成邀请码，其他项目填写邀请码，将两个项目关联起来

or

填写其他项目的邀请码

生成邀请码

邀请码：

生成

或

关联其他项目

请输入要关联的项目生成的邀请码

关联

- 邀请码的有效时间是5分钟，请及时填写
- 目前关联的项目缺陷记录是可以共享的

7. 查看commit记录功能

点击侧边栏第八项**提交记录**，可以查看~~该项目的~~**所有提交记录** 该项目与task和bug操作相关的commit记录